

QUALITY ASSURANCE TESTING DOCUMENTATION (QATD)

UMHackathon 2026

umhackathon@um.edu.my

Table of Content

Document Control	3
PRELIMINARY ROUND (Test Strategy & Planning)	3
1. Scope & Requirements Traceability	3
2. Risk Assessment & Mitigation Strategy	3
3. Test Environment & Execution Strategy	5
4. CI/CD Release Thresholds & Automation Gates	5
5. Test Case Specifications (Drafts)	6
6. Test Strategy & Plan Sign-Off	7

Document Control

Field	Detail
System Under Test (SUT)	UMHackathon 2026 - Team Big Guys
Team Repo URL	https://github.com/FlyingShuriken/homie
Live Deployment URL (optional for preliminary round)	N/A

Objective:

*The primary objective of this testing documentation is to verify that **Homie** can execute its core rental-search workflow reliably: accept search filters, run multi-source listing collection, normalize and score listings, present ranked results, and support inquiry drafting and handoff. A secondary objective is to identify release blockers and operational risks before hackathon judging, especially around scraper stability, AI fallback behavior, and frontend build readiness.*

PRELIMINARY ROUND (Test Strategy & Planning)

1. Scope & Requirements Traceability

1.1 In-Scope Core Features

- Rental Search Submission: User enters location, price range, room type, furnishing preference, transport preference, gender restriction, parking, pet-friendly, and max results.
- Multi-Source Listing Collection: Backend collects listings from ibilik, iProperty, and Facebook-related flows.
- Normalization & Ranking: Raw listings are normalized into a common structure, scored, and returned as ranked results with explanations.
- Progress Visibility: Users can observe the workflow through server-sent events.
- Inquiry Drafting & Handoff: User can request a draft message and continue via Telegram or manual phone fallback.

1.2 Out-of-Scope

- User login and saved accounts
- Scheduled recurring search automation
- Automated sending of inquiry messages
- Booking, payment, or transaction handling
- Full semantic duplicate detection across sources
- Additional sources beyond the currently implemented set

2. Risk Assessment & Mitigation Strategy

Technical Risk	Likelihood (1–5)	Severity (1–5)	Risk Score (L×S)	Mitigation Strategy	Testing Approach
Scraper DOM/API changes break source ingestion	4	5	20 (Critical)	Keep source scrapers isolated; maintain demo-seed fallback for judging	Run seeded smoke tests and manual source-specific checks
Frontend release build blocked by external dependency or environment assumptions	4	4	16 (Critical)	Reduce build-time network dependence; verify production build before release	Run npm run build as a release gate
AI returns malformed JSON or unusable text for explanations/report	3	4	12 (High)	Keep deterministic fallbacks for explanations and reports	Inject AI failures and verify pipeline still completes
No automated backend test suite leads to unnoticed regression	4	4	16 (Critical)	Add minimum smoke and API regression tests to CI	Run pytest and require non-zero executed suites

Duplicate listings reduce trust in ranking	3	3	9 (Medium)	Keep URL-based dedupe now; add semantic duplicate logic later	Verify fixture runs do not explode duplicate count
Incorrect contact-channel interpretation breaks handoff	3	4	12 (High)	Validate Telegram vs phone/manual branches clearly in UI and backend	Test draft + handoff flows for listings without Telegram
Slow or blocked third-party sources produce incomplete results	4	3	12 (High)	Continue with partial results, surface progress events, keep demo mode	Partial-sourc e smoke tests and graceful failure review

Risk Assessment Scoring Criteria:

Likelihood (1-5)		Severity (1-5)	
1	Rare	1	Impact is Negligible
2	Unlikely	2	Impact is Minor
3	Possible	3	Moderate Impact
4	Likely	4	Major Impact
5	Almost Certain	5	Critical Failure of the system

Risk Score = Likelihood × Severity

Risk Level Reference:

Risk Score	Risk Level	Recommended Action
1 – 5	Low	Monitor only. Acceptable risks.
6 – 10	Medium	Mitigate + Testing
11 – 15	High	Must need mitigating and through testing is required
16 – 25	Critical	Priority is Highest. Need extensive level of testing.

3. Test Environment & Execution Strategy

Explain the testing environment. That means where the testing of your application will take place, how you are going to handle test data and the rules/threshold that you are incorporating to govern your testing phases.

- **Unit Test**
 - **Current Scope:** No committed backend unit test suite is present yet.
 - **Execution:** `uv run pytest -q`
 - **Isolation:** N/A in current repo state because no executable tests were collected.
 - **Pass Condition:** At least one maintained automated suite should execute; current state does not meet this.
 -
- **Integration Test**
 - **Scope:** Search session creation, orchestrator-driven tool execution, result persistence, outreach draft generation, and handoff fallback
 - **Execution:** Controlled backend integration smoke tests using FastAPI TestClient
 - **Workflow:** Temporary SQLite database, `DEMO_SEED=true`, deterministic mocked AI responses, and repository fixture listings
 - **Pass Condition:** Pipeline reaches complete, returns listings, produces summary text, and allows inquiry draft generation without crashing
- **Test Environment (CI/CD Practice):**
 - **Local Backend QA:** FastAPI app invoked through TestClient
 - **Data Store:** Temporary SQLite database under `/tmp/` for deterministic runs
 - **Test Data:** `backend/tests/fixtures/ibilik_seed.json` and `backend/tests/fixtures/iproperty_seed.json`
 - **AI Layer:** Deterministic mocked AI responses for workflow contract testing; failure-injection variant for fallback verification
 - **Sandbox Constraint:** Outbound network unavailable in this environment, which affects live model calls and Google Fonts fetching during frontend build
- **Regression Testing & Pass/Fail Rules:**
 - **Execution Phase:** Run core smoke cases after meaningful backend or frontend changes
 - **Pass/Fail Condition:** Critical flow is only considered passed if the session reaches completion, results are retrievable, and inquiry support works
 - **Continuation Rule:** Release should not proceed if critical smoke flow fails, if no automated tests execute, or if production build is blocked
- **Test Data Strategy:**
 - **Manual / Seeded:** Use curated fixture listings that represent multi-source Cheras rental inventory
 - **Runtime Generated:** Session IDs, temporary SQLite records, and generated inquiry drafts
 - **Controlled AI Contracts:** Deterministic mocked responses to validate workflow sequencing and fallback behavior

- **Passing Rate Threshold:**
 - **Critical flow tests:** 100% pass required
 - **Overall executed QA checks:** 85% minimum target
 - **Build gate:** frontend production build must succeed
 - **Automation gate:** at least one backend automated suite must execute

4. CI/CD Release Thresholds & Automation Gates

4.1 Integration Thresholds (Merging to Main)

The primary needs of this threshold to be crucial is because the main branch must be a stable source of truth. So, it needs to be achieved through continuous integration checks.

Checks	Requirements	Project	Pass/Failed
Backend Smoke Import	Zero import/runtime init error	Passed: backend app imports and initializes	Passed
Automated Backend Tests	At least 1 executable maintained suite	0 tests collected via uv run pytest -q	Failed
Critical Backend Flow	Search -> results -> draft/handoff works in controlled QA run	Passed: pipeline completed with 14 listings and inquiry draft path	Passed
Code Quality / Build Readiness	Frontend production build succeeds	Failed: npm run build blocked by Google Fonts fetch in sandbox	Failed

<i>Fallback Safety</i>	<i>AI explanation/report failure does not crash pipeline</i>	<i>Passed: fallback explanation and report remained available</i>	<i>Passed</i>
-------------------------------	---	--	----------------------

4.2 Deployment Thresholds (Pushing to Production)

Checks	Requirements	Project	Pass/Failed
Regression Core Flow	100% of critical smoke scenarios must pass	4 / 4 critical backend scenarios passed in controlled harness	Passed
Frontend Release Build	Successful production build	Blocked by external Google Fonts fetch failure	Failed
AI Output Contract	Generated outputs must meet acceptance criteria or trigger safe fallback	Passed in controlled harness and failure-injection test	Passed
Critical Bugs	Zero release-blocking defects	At least 2 blockers remain: no automated backend suite, frontend build gate not green	Failed
API Readiness (Local Harness)	Stable endpoint behavior under repeated result fetch	Average GET /results latency: 1.66 ms over 20 local samples	Passed

5. Test Case Specifications

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
TC-01	Happy Case (Entire Flow): Search -> rank -> inquiry	Verify a renter can start a search, complete the seeded pipeline, retrieve ranked listings, and generate an inquiry draft	1. Start backend in controlled QA harness with temp SQLite, DEMO_SEED=true, mocked AI responses. 2. POST /api/search with Cheras filters. 3. Poll /api/search/{id}/results. 4. Request /api/outreach/draft for top listing.	Session completes, listings are persisted, summary report is returned, ranked results are visible, and draft generation succeeds.	Passed. Session completed, 14 listings were persisted, summary report returned successfully, top listing was "Single Room @ Taman Connaught, Cheras" with score 75.8 , and inquiry draft generation succeeded.
TC-02	Specific Case (Negative): Invalid filter validation	Verify the system rejects an invalid price range without creating a broken search session	1. POST /api/search with price_min=900 and price_max=500. 2. Observe API response.	Request is rejected with clear validation feedback; no uncontrolled crash.	Passed. Backend returned HTTP 422 with message: "price_max must be greater than price_min."

TC-03	NFR (Performance): Results endpoint responsiveness	Verify repeated reads of completed results remain fast in local QA conditions	1. Complete controlled QA search session. 2. Call <code>/api/search/{id}/results</code> 20 times. 3. Record response timings.	Average response time remains below local threshold and no request errors occur.	Passed. Average response time was 1.66 ms , maximum 2.11 ms , with 0% error rate in the local harness.
TC-04	NFR (Build / Release Readiness): Frontend production build	Verify the frontend can be built for release without blocking dependency failures	1. Run <code>npm run build</code> in frontend/. 2. Observe completion status.	Production build completes successfully.	Failed. Build stopped with next/font fetch failures for Space Grotesk , Instrument Serif , and JetBrains Mono because the environment could not resolve <code>fonts.googleapis.com</code> .
TC-05	NFR (Boundary / Capacity): Oversized user request handling	Verify a very high requested result count does not cause uncontrolled behavior	1. Submit search with <code>max_results=200</code> . 2. Run seeded pipeline. 3. Retrieve results.	System remains bounded, completes successfully, and returns only available/allowed data without crashing.	Passed. Pipeline completed successfully and returned 14 listings from seeded sources without instability.

TC-06	Specific Case (Fallback Branch): Manual contact handoff	Verify a listing without Telegram can still proceed through a manual contact path	1. Generate inquiry draft for listing without Telegram. 2. POST /api/outreach/hand off.	System returns manual phone fallback instead of failing the flow.	Passed. Backend returned phone fallback 0123456789 for the tested listing.
--------------	---	---	---	---	---

6. AI Output & Boundary Testing

Execution Note: In the current sandbox, live online model benchmarking was not feasible because outbound network access is restricted. AI-output testing below therefore focused on:

- prompt/response contract validation in a controlled harness,
- correct response shape,
- safe fallback behavior under failure, and
- bounded-input handling.

6.1. Prompt/Response Test Pairs

Test ID	Prompt Input	Expected Output (Acceptance Criteria)	Actual Output	Status
AI-01	Generate final session summary from aggregated search stats	2-4 concise sentences; must mention listing count or search context; must not invent unsupported facts	"Found 10 listings for Cheras. Most sit within the requested budget and the top results align well with the chosen area and room type."	Pass

AI-02	Generate score explanations for a batch of ranked listings	Must return valid JSON mapping listing_id -> explanation, with no markdown fencing and usable explanatory text	Example accepted explanation: "Single Room @ Taman Connaught, Cheras matches the requested area and budget reasonably well from the available data."	Pass
AI-03	Generate inquiry draft for selected rental listing	Polite rental inquiry, short and editable, no fabricated promises or facts	"Hello, I am interested in this room and would like to know if it is still available. Could you please share the latest details and viewing arrangement? Thank you."	Pass

6.2. Oversized/Larger Input Test

In this section, do define the maximum input size your system accepts. Do highlight here what happens when the limit is exceeded.

Fields	Details
Maximum Input Size	Practical request bound is enforced by application behavior rather than free-text token limits; per-source scraper collection is capped by MAX_LISTINGS_PER_SOURCE (default 25)
Input used while testing	Search request with max_results=200
Expected Behavior	Request should remain bounded; system should not crash or attempt unbounded downstream processing

Actual Behavior	Pipeline completed successfully and returned 14 fixture listings from the available seeded sources
Status	Pass

6.3. Adversarial/Edge Prompt Test

Scenario: Malformed user input at the search boundary

- **Input:** valid location with invalid numeric range (price_min > price_max)
- **Expected Handling:** reject request cleanly before pipeline creation
- **Actual Handling:** backend returned **HTTP 422** with explicit field-level validation error on price_max
- **Status:** Pass

6.4. Hallucinating Handling

Homie currently mitigates hallucination and malformed AI output in a few practical ways:

- Numeric listing scoring is deterministic backend code, not model-generated.
- Score explanations fall back to a rule-based explanation if the model returns unusable output.
- Session summary report falls back to a template report if model generation fails.
- Inquiry drafting falls back to a safe generic message if the model call fails.
- Prompt inputs are trimmed and structured to reduce opportunities for unstable free-form output.

Failure-Injection Verification Result

Test ID	Failure Injected	Expected Behavior	Actual Behavior	Status
AI-04	Score explanation generation throws exception	Pipeline completes; fallback explanation remains present	Passed. First listing explanation fell back to " Score 75.8/100 - in target area "	Pass

AI-05	Summary report generation throws exception	Pipeline completes; template report is returned	Passed. Summary fell back to "Found 14 rental listings in Cheras. Average price: RM 640/month. Top match score: 75.8/100."	Pass
--------------	--	---	--	-------------

Important Note:

Using Agile principles, QA for Homie should remain iterative. The current document reflects a meaningful but still incomplete test baseline. Before hackathon judging or any wider release, the project should prioritize:

- 1. adding committed automated backend tests,*
- 2. making frontend builds independent of blocking external font fetches, and*
- 3. repeating core smoke tests against live scraper and live GLM conditions in a connected environment.*